

Query Processing

Join Operations: Nested-Loop Algorithms

1. Introduction

The **join operation** is one of the most expensive operations in query processing. An **equi-join** has the form $r \bowtie_{r.A=s.B} s$, where A and B are attributes of relations r and s .

Key Terminology

- n_r, n_s : Number of tuples in relations r and s
- b_r, b_s : Number of blocks containing tuples of r and s
- M : Number of memory buffer blocks available
- **Outer relation**: The relation in the outer loop
- **Inner relation**: The relation in the inner loop (scanned repeatedly)
- t_S : Seek time (typically 4 ms for HDD)
- t_T : Block transfer time (typically 0.1 ms for HDD)

1.1 Running Example

Throughout this document, we use the join: `student` $\bowtie$ `takes`

Relation	Tuples (n)	Blocks (b)
<code>student</code>	$n_{student} = 5,000$	$b_{student} = 100$
<code>takes</code>	$n_{takes} = 10,000$	$b_{takes} = 400$

2. Nested-Loop Join (Basic)

The simplest join algorithm: for each tuple in r , scan the entire relation s .

Algorithm: Nested-Loop Join

```
for each tuple t_r in r do:
  for each tuple t_s in s do:
    if (t_r, t_s) satisfy join condition theta:
      add t_r · t_s to result
```

Characteristics:

- Requires **no indices** — works for any join condition

- Very expensive: examines $n_r \times n_s$ tuple pairs
- The inner relation s is scanned once for **each tuple** in r

Cost Formula: Nested-Loop Join

Worst Case (only 1 block of each relation fits in memory):

$$\text{Block Transfers} = n_r \times b_s + b_r$$

$$\text{Seeks} = n_r + b_r$$

Best Case (both relations fit in memory):

$$\text{Block Transfers} = b_r + b_s \quad \text{Seeks} = 2$$

Formula Breakdown (Worst Case):

1. $n_r \times b_s$: For each of the n_r tuples in r , we scan all b_s blocks of s
2. $+b_r$: We also need to read each block of r once
3. n_r seeks: One seek per tuple in r to restart the s scan
4. $+b_r$ seeks: Seeks needed to read blocks of r

Example: student $\bowtie$ takes (Nested-Loop)

Configuration: student is outer, takes is inner

Tuple Pairs Examined: $5,000 \times 10,000 = 50,000,000$ pairs

Worst Case Cost:

$$\text{Block Transfers} = 5,000 \times 400 + 100 = \mathbf{2,000,100}$$

$$\text{Seeks} = 5,000 + 100 = \mathbf{5,100}$$

Best Case Cost:

$$\text{Block Transfers} = 100 + 400 = \mathbf{500} \quad \text{Seeks} = \mathbf{2}$$

Alternative: If takes is outer, student is inner:

$$\text{Block Transfers} = 10,000 \times 100 + 400 = \mathbf{1,000,400}$$

Lower block transfers, but more seeks (10,400).

Tip: If one relation fits entirely in memory, use it as the **inner relation**. The inner relation is then read only once, giving best-case performance.

3. Block Nested-Loop Join

Instead of iterating tuple-by-tuple, iterate **block-by-block**. This dramatically reduces I/O.

Algorithm: Block Nested-Loop Join

```
for each block B_r of r do:
  for each block B_s of s do:
    for each tuple t_r in B_r do:
      for each tuple t_s in B_s do:
        if (t_r, t_s) satisfy join condition:
          add t_r · t_s to result
```

Key Improvement: Each block of s is read once per **block** of r (not per tuple).

Cost Formula: Block Nested-Loop Join

Worst Case:

$$\text{Block Transfers} = b_r \times b_s + b_r$$

$$\text{Seeks} = 2 \times b_r$$

Best Case (inner relation fits in memory):

$$\text{Block Transfers} = b_r + b_s \quad \text{Seeks} = 2$$

Formula Breakdown:

1. $b_r \times b_s$: For each of the b_r blocks in r , scan all b_s blocks of s
2. $+b_r$: Read each block of r once
3. $2 \times b_r$ seeks: One seek per block of r + one seek per scan of s

Example: student $\bowtie$ takes (Block Nested-Loop)

Configuration: student is outer, takes is inner

Worst Case Cost:

$$\text{Block Transfers} = 100 \times 400 + 100 = \mathbf{40,100}$$

$$\text{Seeks} = 2 \times 100 = \mathbf{200}$$

Comparison with Basic Nested-Loop:

Metric	Nested-Loop	Block Nested-Loop
Block Transfers	2,000,100	40,100
Seeks	5,100	200

Improvement: $\approx 50\times$ fewer block transfers!

3.1 Optimization: Use Larger Outer Blocks

Instead of reading one block of r at a time, read $(M - 2)$ blocks:

Optimized Block Nested-Loop Cost

$$\text{Block Transfers} = \left\lceil \frac{b_r}{M-2} \right\rceil \times b_s + b_r$$

$$\text{Seeks} = 2 \times \left\lceil \frac{b_r}{M-2} \right\rceil$$

Why $M - 2$? Reserve 1 block for inner relation buffer, 1 for output buffer.

Example with $M = 52$ blocks

$$\text{Block Transfers} = \left\lceil \frac{100}{50} \right\rceil \times 400 + 100 = 2 \times 400 + 100 = \mathbf{900}$$

$$\text{Seeks} = 2 \times 2 = \mathbf{4}$$

Massive improvement from 40,100 transfers to just 900!

4. Indexed Nested-Loop Join

If an index exists on the join attribute of the inner relation, use it instead of scanning.

Algorithm: Indexed Nested-Loop Join

```
for each tuple t_r in r do:
    use index on s to find tuples where s.B = t_r.A
    for each matching tuple t_s:
        add t_r · t_s to result
```

Key Idea: Replace the inner loop's full scan with an **index lookup**.

Cost Formula: Indexed Nested-Loop Join

$$\text{Cost} = b_r \times (t_T + t_S) + n_r \times c$$

Where c = cost of one index lookup + retrieving matching tuples from s .

For a B⁺-tree with height h_i on a key attribute:

$$c = (h_i + 1) \times (t_T + t_S)$$

Formula Breakdown:

1. $b_r \times (t_T + t_S)$: Read all blocks of the outer relation r
2. $n_r \times c$: For each of the n_r tuples in r , perform an index lookup on s

Example: student ⋈ takes (Indexed Nested-Loop)

Configuration:

- **student** is outer relation
- **takes** has a clustering B⁺-tree index on ID
- Index has 20 entries per node $\Rightarrow$ height $h_i = 4$

Cost Calculation:

$$c = (4 + 1) = 5 \text{ disk accesses per lookup}$$

$$\text{Total} = 100 + 5,000 \times 5 = \mathbf{25,100} \text{ disk accesses}$$

Each disk access = 1 seek + 1 transfer.

Comparison:

Algorithm	Disk Accesses
Block Nested-Loop	40,100 transfers + 200 seeks
Indexed Nested-Loop	25,100 accesses (each = seek + transfer)

Trade-off: Fewer block transfers but more seeks. Indexed version is better when the outer relation is small or highly selective.

When to use Indexed Nested-Loop:

- Index exists on inner relation's join attribute
- Outer relation is small (or filtered by a selection)
- Join attribute is a key (each lookup returns few tuples)

5. Summary and Comparison

Algorithm	Transfers (Worst)	Seeks (Worst)	When to Use
Nested-Loop	$n_r \times b_s + b_r$	$n_r + b_r$	Last resort; no index available
Block Nested-Loop	$b_r \times b_s + b_r$	$2 \times b_r$	No index; relations don't fit in memory
Optimized Block	$\lceil b_r / (M - 2) \rceil \times b_s + b_r$	$2 \times \lceil b_r / (M - 2) \rceil$	Large memory available
Indexed Nested-Loop	$b_r + n_r \times c$	(included in c)	Index on inner join attribute

Key Takeaways

- **Basic Nested-Loop** is extremely expensive: $O(n_r \times b_s)$
- **Block Nested-Loop** improves to $O(b_r \times b_s)$ — use smaller relation as outer
- **With more memory**, read $(M - 2)$ blocks of outer at a time
- **With an index**, Indexed Nested-Loop can be fastest for selective joins
- Always choose the **smaller relation** as outer when using indexed version